1.-Introducción

Como se ha comentado a lo largo de los contenidos de esta asignatura, un sistema ERP puede entenderse como un puzzle. Un gran puzzle modular que permite la creación de nuevos módulos que pueden modificar o ampliar las funcionalidades del ERP original. Módulos que se interconectan unos con otro, ayudándonos en nuestro día a día. Módulos que definen la funcionalidad del ERP.
Es por ello que hemos de conocer cómo crear módulos en nuestro sistema ERP, algo que nos da una gran ventaja dado que podemos re-aprovechar aplicaciones, crear nuevas soluciones… y sobre todo trabajar con mayor autonomía en nuestro sistema ERP: **saberlo adaptar más y mejor a nuestras necesidades concretas.**

Este tema se divide en los siguientes apartados:

1. Introducción
1.1. Técnicas y estándares. Modelo vista controlador
2. Lenguaje proporcionado por los sistemas ERP
2.1. Características y sintaxis del lenguaje
2.2. Declaración de datos. Estructuras de programación. Sentencias del lenguaje.
2.3. Llamadas a funciones
2.4. Librerías de funciones (APIs)
3. Entornos de desarrollo y herramientas de desarrollo en sistemas ERP.
3.1. Depuración de un programa
3.2. Manejo de errores.
4. Creación de módulos con Odoo
4.1. PASO 1: análisis tecnológico inicial
4.2. PASO 2: desarrollar una serie de modelos de datos
4.2.1. Crear un módulo vacío.
4.2.2. Edición inicial ficheros del módulo
4.2.3. Creación de modelos Python
4.3. PASO 3: mapear los modelos de datos en Odoo.
4.4. PASO 4: crear vistas y menús para visualizar los modelos de datos.
4.4.1. Creación archivos XML
4.4.2. Completar ficheros XML
4.4.3. Crear una acción
4.4.4. Crear menús de acceso.

Apartados que pasan a detallarse a continuación.

Para saber más

Desde un punto de vista comercial, el desarrollo de módulos en Odoo, a menudo se conoce como “desarrollo de aplicaciones”. Odoo dispone de un página web donde diferentes desarrolladores incluyen las aplicaciones que han creado: https://www.odoo.com/apps

Esto es algo que suele existir en otros sistemas ERP, no obstante, en Odoo es especialmente potente debido a sus orígenes como comunidad de

software libre.

Además, que sea software libre no debe confundirse con el término “gratis”. En el enlace anterior puedes observar cómo los desarrolladores fijan diferentes precios para cada aplicación de forma similar a como ocurre en tiendas de aplicaciones generalistas como la Play Store o el App Store de Apple.

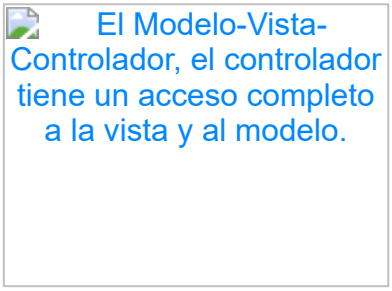


Ministerio de Educación, Formación Profesional y Deportes

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

Aviso Legal

1.1.- Técnicas y estándares. Modelo-Vista-Controlador.


El Modelo-Vista-Controlador, el controlador tiene un acceso completo a la vista y al modelo.

¿Utilizas habitualmente hojas de cálculo, como por ejemplo la de LibreOffice? En ese caso, sabrás que al introducir datos en las celdas, los mismos datos podemos verlos de varias formas. Es posible ver los datos en la hoja de cálculo donde los hemos introducido, o mediante un gráfico que puede ser de barras, circular, etc.

Esos programas implementan **el patrón Modelo-Vista-Controlador (MVC).** MVC son las siglas de "Modelo-vista-controlador" es un patrón de arquitectura de software, que separa los datos y la lógica de negocio de una aplicación de su representación y el módulo encargado de gestionar los eventos y las comunicaciones.

En el ejemplo de la hoja de cálculo, el modelo lo constituyen los datos que hemos introducido en las celdas. Las vistas se encargan de mostrar los datos de la forma que se seleccione.

En resumen, el MVC divide una aplicación en tres componentes:

- Los datos de la aplicación (**modelo**).
- La interfaz del usuario (**vista**).
- El **controlador**, el cual define la forma en que la interfaz reacciona a la entrada del usuario.

Con esto se consigue separar los datos (modelo) de la interfaz de usuario (vista), de manera que los cambios en la interfaz no afectan a los datos y viceversa, es decir, que los datos pueden ser cambiados sin afectar a la interfaz de usuario.

En Odoo, el MVC se implementa de la siguiente forma:

- El **modelo** son las tablas de la base de datos.
- La **vista** son los archivos XML que definen la interfaz de usuario del módulo.
- El **controlador** son los objetos creados en Python.

Por ejemplo, en el diagrama anterior, las flechas continuas significan que el controlador tiene un acceso completo a la vista y al modelo, y las flechas discontinuas significan un acceso limitado. Las razones de este diseño son las siguientes:

- Acceso de **modelo** a **vista**: el modelo envía una notificación a la vista cuando sus datos han sido modificados, con el fin de que la vista pueda actualizar su contenido. El modelo no necesita conocer el funcionamiento interno de la vista para realizar esta operación. Sin embargo, la vista necesita acceder a la partes internas del controlador.
- Acceso de **vista** a **controlador**: la razón de que la vista tenga limitado el acceso al controlador es porque las dependencias que la vista tiene sobre el controlador deben ser mínimas ya que el controlador puede ser sustituido en cualquier momento.

Autoevaluación

Pregunta

En el modelo-vista-controlador la vista son los archivos XML, el modelo las tablas y el controlador el código que define el comportamiento de la aplicación.

Respuestas

Verdadero.

Falso.

Retroalimentación

Correcto. Es la separación entre los componentes de la aplicación que establece el modelo-vista-controlador.

Incorrecto. Es la separación entre los componentes de la aplicación que establece el modelo-vista-controlador.

Solución

1. Opción correcta
2. Incorrecto

2.- Lenguaje proporcionado por los sistemas ERP

Como se ha comentado en los contenidos de esta asignatura, la mayoría de sistemas ERP están construidos con lenguajes de programación modernos y, en muchos casos, orientados a objetos. Existen aplicaciones ERP que proporcionan lenguajes propietarios, que no son lenguajes estándar y requieren que el programador se forme en ese lenguaje sólo y exclusivamente para manejar el ERP.

El lenguaje de programación utilizado en Odoo es Python. Es por ello que los modelos de datos en Odoo se llevan a cabo utilizando el lenguaje de programación Python. Será importante familiarizarse con el mismo para poder cumplir satisfactoriamente los contenidos de esta unidad.



El lenguaje Python fue creado por Guido van Rossum a principios de los años 90 en el Centro para las Matemáticas y la Informática (CWI, Centrum Wiskunde & Informatica), en los Países Bajos.

En 2001, se creó la Python Software Foundation (PSF), una organización sin fines de lucro creada específicamente para poseer la propiedad intelectual sobre la licencia y promover el uso del lenguaje. Como miembros patrocinadores de esta organización están Google, Canonical, Sun Microsystem y Microsoft.

La licencia de Python es de código abierto, compatible con GPL. Es un lenguaje muy sencillo de utilizar con una sintaxis muy cercana al lenguaje natural, por lo que se considera uno de los mejores lenguajes para empezar a programar.

Python es utilizado por **Google, Yahoo, la NASA** o por ejemplo en todas las distribuciones Linux, donde los programas escritos en este lenguaje representan un porcentaje cada vez mayor.

.

Debes conocer

Dos curiosidades a apuntar sobre Python:

1. El nombre del lenguaje Python proviene de la afición de su creador original por el grupo de cómicos ingleses "Monty Python".

2.

Como muchos desarrolladores afirman “a Python o lo odias o lo amas, no hay término medio”. La simplicidad de la sintaxis de Python irrumpió con tal fuerza cuando fue formulado, que dividió las opiniones entre los desarrolladores: para unos es genial que sea tan sencilla mientras que otros consideran que tanta simplicidad hacer perder “la esencia” a la tarea de programar. Y tú, ¿qué opinas?

Para saber más

Para saber más sobre Python, se recomienda acceder a su web oficial:

Web oficial del lenguaje Python.

Además, con una simple búsqueda por Internet podrás encontrar información más que útil sobre Python, como comunidades o blogs sobre el tema.

2.1.- Características y sintaxis del lenguaje.

Las principales características de Python son:



- **Sintaxis muy sencilla**, lo cual facilita un rápido aprendizaje del lenguaje.
- **Lenguaje interpretado**. Los programas en Python se ejecutan utilizando un programa intermedio llamado intérprete. El código fuente se traduce a un archivo llamado **bytecode** con extensión **.pyc** o **.pyo**, que es el que se ejecutará en sucesivas ocasiones.
- **Tipado dinámico**. Esta característica quiere decir que los datos no se declaran antes de utilizarlos, sino que el tipo de una variable se determina en tiempo de ejecución, según el dato que se le asigne.
- **Fuertemente tipado**. Cuando una variable es de un tipo concreto, no se puede usar como si fuera de otro tipo, a no ser que se haga una conversión de tipo. Si la variable es numérica entera, por ejemplo, no podré asignarle un valor real. En otros lenguajes, el tipo de la variable cambiaría para adaptarse al comportamiento esperado, aunque ello puede dar más lugar a errores.
- **Multiplataforma**. El intérprete de Python está disponible para una gran variedad de plataformas: Linux, Windows, Macintosh, etc.
- **Orientado a objetos**. Los programas están formados por clases y objetos, que son representaciones del problema en el mundo real. Los datos, junto con las funciones que los manipulan, son parte interna de los objetos y no están accesibles al resto de los objetos. Por tanto, los cambios en los datos de un objeto sólo afectan a las funciones definidas para ese objeto, pero no al resto de la aplicación.

Para hacer el primer programa en Python no necesitamos conocer mucho sobre el lenguaje, por ejemplo, podemos hacer el famoso **"Hola mundo"** con el que se suele comenzar a estudiar casi todos los lenguajes, simplemente con la siguiente línea:
print ('Hola mundo')

Si escribimos esto en el intérprete de Python se ejecutará y se imprimirá por pantalla el mensaje **"Hola mundo"**. ¡Ya tenemos nuestro primer programa!. Probémoslo. El intérprete de Python está instalado por defecto en la mayoría de las distribuciones de Linux, para saber si lo tenemos instalado simplemente tenemos que introducir phyton en un terminal, y nos aparecerá un mensaje con la versión que tenemos instalada y el prompt >>>, que indica que está esperando código del usuario. Podemos salir escribiendo **exit()**, o con la combinación de teclas **Control + D**.

También podemos guardar el código en un archivo llamado **programa.py**, y ejecutarlo con la orden python **programa.py**, el resultado será que se escribirá la cadena de texto **'Hola mundo'**.

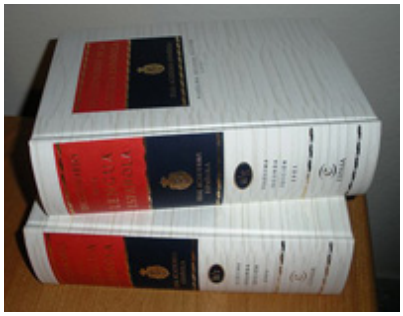
Los comentarios en el lenguaje se escriben con el carácter #, las asignaciones de valores a las variables con el carácter igual (=). Para crear una función utilizamos el siguiente código, hay que tener en cuenta que la sangría del texto sirve para delimitar qué instrucciones van dentro de la función:

```
def nombrefunción(arg1,arg2..):
instrucción1
instrucción2
...
```

instrucciónN

Es decir, en este lenguaje no hay delimitador de sentencia como el punto y como de Java o C, ni delimitador de bloques, como las llaves de estos mismos lenguajes.

2.2.- Declaración de datos. Tipos básicos.



Como ya hemos comentado, en Python no se declaran las variables como tal, sino que se utilizan directamente. Podemos declarar variables de los siguientes tipos básicos:

- **Números enteros**. Los números enteros son el cero y aquellos números positivos o negativos que no tienen decimales. En Python se pueden representar mediante el tipo **int** que utiliza 32 o 64 bits, dependiendo del tipo de procesador, o el tipo **long**, que permite almacenar números de cualquier precisión, estando limitados solo por la memoria disponible en la máquina.

```
# declaración de un número entero
a = 2
# declaración de un número entero largo
a = 2L
```

- **Números reales**. Son los que tienen la coma decimal. Se expresan mediante el tipo **float**.

```
b = 2.0
```

- **Números complejos**. Son un tipo de dato especial compuesto por una parte real y una parte imaginaria.

```
complejo = 2.0 + 7.5 j
```

- **Cadenas**. Es texto encerrado entre comillas simples o dobles.

```
mensaje = "Bienvenido a Python"
```

- **Booleanos**. Sólo pueden tener dos valores **True** o **False**.

De entre los operadores más importantes para trabajar con estos datos podemos destacar los siguientes:

- **Operadores aritméticos**: suma (+), resta (-), multiplicación (*), división(/), división entera (//), exponente (**), módulo (%).
- **Operadores booleanos**: **and, or, not.**
- **Operadores relacionales**: igual (==), distinto (!=), menor (<), mayor (>), menor igual(<=), mayor o igual (>=).

Autoevaluación

Pregunta

En Python la representación de números enteros depende de la máquina donde se esté ejecutando el programa.

Respuestas

Verdadero.

Falso.

Retroalimentación

Correcto. El tipo int de Python se implementa a bajo nivel mediante un tipo long de C, y dado que Python utiliza C por debajo, como C, el rango de los valores que puede representar depende de la plataforma.

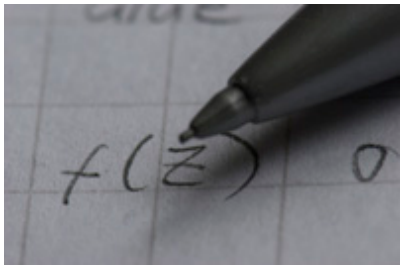
Incorrecto. El tipo int de Python se implementa a bajo nivel mediante un tipo long de C, y dado que Python utiliza C por debajo, como C, el rango de los valores que puede representar depende de la plataforma.

Solución

1. Opción correcta
2. Incorrecto

2.3.- Llamadas a funciones.

Una función es un conjunto de instrucciones o programa que realiza una tarea específica y que devuelve un valor. Las funciones nos ayudan a reutilizar código, y a dividir el programa en partes más pequeñas con objeto de hacerlo más organizado, legible y más fácil de depurar.




```
def   cuadrado ( numero ):
    print (numero ** 2)
```

Con el código anterior estamos declarando la función **cuadrado** que, como su nombre indica, calcula el cuadrado de un número. El número que aparece entre paréntesis es el valor que le pasamos a la función para que pueda calcular su cuadrado, y recibe el nombre de **parámetro**. Las funciones pueden tener uno, varios o ningún parámetro.

Cuando ejecutamos la función se dice que estamos haciendo una llamada a la función, y lo hacemos utilizando su nombre y los parámetros entre paréntesis:

```
cuadrado (4)
```

El resultado de llamar a la función será que se imprimirá por pantalla el valor **16**.

Las funciones pueden o no devolver un valor como resultado de su ejecución. Podemos transformar el ejemplo anterior para que devuelva el resultado en lugar de mostrarlo por pantalla. El código sería el siguiente:

```
def   cuadrado ( numero ):
    return numero ** 2
```

Y llamaríamos a la función con la siguiente instrucción:

```
print (cuadrado(4))
```

El resultado sería el mismo, sólo que ahora la función devuelve el valor, y ese valor es el que muestra por pantalla la instrucción **print**.

Autoevaluación

Pregunta

Las funciones en Python, al igual que los procedimientos en otros lenguajes, a veces no devuelven ningún valor.

Respuestas

Verdadero.

Falso

Retroalimentación

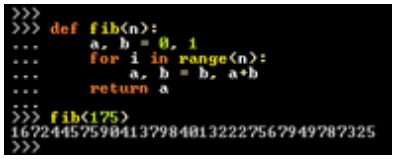
Incorrecto. Las funciones siempre devuelven un valor, y cuando el programador no especifica un valor de retorno la función devuelve el valor None.

Correcto. Las funciones siempre devuelven un valor, y cuando el programador no especifica un valor de retorno la función devuelve el valor None.

Solución

1. Incorrecto
2. Opción correcta

2.4.- Librerías de funciones (APIs).



Una **API**, o **Biblioteca de Clases**, es un conjunto de clases útiles que tienen a disposición los programadores para utilizar en sus programas. Python proporciona una API estándar, cuyo código fuente está a libre disposición para las principales plataformas desde el sitio web de Python. También es posible acceder a muchos módulos libres de terceros en la misma página web, así como a programas, herramientas y documentación adicional.

En el enlace expresado más abajo encontrarás una referencia a toda la Biblioteca estándar de módulos de Python, entre los que destacan:

- **os:** provee de diferentes funciones del sistema operativo, como creación de archivos y directorios, leer o escribir de un archivo, manipular rutas, etc.
- **sys:** permite acceder a información sobre el intérprete de Python, como por ejemplo el prompt del sistema, datos de licencia y en general cualquier parámetro o variable que tenga que ver con el intérprete.
- **datetime:** módulos para la manipulación de fechas y horas.
- **math:** funciones matemáticas.

Debes conocer

La siguiente imagen muestra un ejemplo de utilización de las librerías Python **os** y **time**

```
import os
import time
def obtener_informacion_sistema():
    # Obtener el nombre de usuario actual del sistema
    usuario = os.getlogin()

    # Obtener la hora actual del sistema
    hora_actual = time.strftime("%H:%M:%S")

    # Imprimir la información obtenida
    print("Usuario actual:", usuario)
    print("Hora actual del sistema:", hora_actual)

if __name__ == "__main__":
    obtener_informacion_sistema()
```

Para saber más

Para mayor información puedes consultar el siguiente enlace donde se describe la Biblioteca estándar que se distribuye con Python:

Biblioteca de clases estándar de Python.

Autoevaluación

Pregunta

La función chdir() que cambia de directorio la podemos encontrar en el paquete sys.

Respuestas

Verdadero.

Falso.

Retroalimentación

Incorrecto. La función chdir() se encuentra en el paquete os, de manipulación de funciones del sistema operativo.

Correcto. La función chdir() se encuentra en el paquete os, de manipulación de funciones del sistema operativo.

Solución

1. Incorrecto
2. Opción correcta

3.- Entornos de desarrollo en sistemas ERP

La herramienta más importante cuando estamos programando siempre es un buen entorno de desarrollo. Si queremos una respuesta rápida y el programa es pequeño podemos utilizar el intérprete. Sin embargo, cuando el programa va siendo más grande, es mucho más cómodo utilizar un entorno de desarrollo desde el cual podemos hacer todas las operaciones de manera gráfica.

> Un entorno de desarrollo es un programa que incluye todo lo necesario para programar en uno o varios lenguajes. Además de un editor de textos especializado como herramienta central, incluyen navegadores de archivos, asistentes de compilación, depuradores, etc.

Cuando hablamos de entornos de desarrollo, es imposible no pensar en **Eclipse**.Eclipse es una plataforma de software compuesto por un conjunto de herramientas de programación de código abierto multiplataforma para desarrollar lo que el proyecto llama "Aplicaciones de Cliente Enriquecido", opuesto a las aplicaciones "Cliente-liviano" basadas en navegadores. Esta plataforma, típicamente ha sido usada para **desarrollar IDEs. IDE)**



Aunque Eclipse es más que completo, para el desarrollo de las prácticas de esta unidad utilizaremos **Pycharm**.



Pycharm es recomendado para trabajar con Python básicamente porque es **un IDE que está orientado al 100% al desarrollo de aplicaciones Python.** Está desarrollado por la empresa checa JetBrains.

Para saber más

En el siguiente vídeo, puedes conocer como instalar PyCharm en Ubuntu:
LINK VIDEO

3.1.- Depuración de un programa.

Las herramientas de depuración cuando estamos desarrollando un programa ayudan a detectar errores en el código, mostrando información de qué tipo de error es o porqué se produce. Las herramientas de depuración suelen formar parte de los entornos de desarrollo.

El lenguaje Python incorpora un depurador dentro de su Biblioteca de módulos estándar llamado pdb. Soporta puntos de ruptura y ejecución paso a paso del código, inspeccionar valores de variables y otras opciones de depuración. Sin embargo, como en todos los lenguajes, si utilizamos una herramienta gráfica para el proceso de depuración, seguramente ahorraremos mucho tiempo.

Dentro del entorno de desarrollo **PyCharm** podemos depurar nuestros programas. Para activar el módulo de depuración hacemos clic en el menú Debug/Debugger. Una vez que esté el modo de depuración activado el funcionamiento es como el de cualquier depurador. Con el botón derecho establecemos una parada en la sentencia del código que queremos evaluar, de manera que al ejecutar el programa el depurador se parará en esa instrucción, y podremos analizar el valor de las variables en ese punto.

Las acciones que podemos realizar en prácticamente cualquier depurador son las siguientes:

- **Go:** hace que la ejecución del programa continúe hasta que encuentre un punto de ruptura.
- **Step:** va ejecutando el programa línea a línea.
- **Over:** hace que la sentencia actual sea completamente ejecutada sin parar en ninguna función anidada.
- **Out:** hace que se compute desde el punto actual hasta el fin de la función actual y que esta se termine.
- **Quit:** finaliza la ejecución del programa.

Autoevaluación

Pregunta

Los programas de depuración por definición siempre se distribuyen junto con los entornos de desarrollo.

Respuestas

Verdadero.

Falso.

Retroalimentación

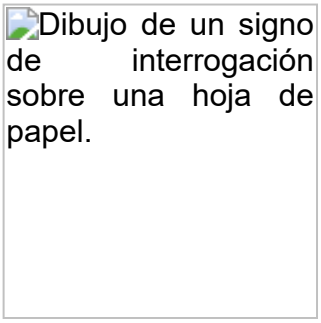
Aunque lo más común es encontrarlos dentro de los entornos de desarrollo, existen programas que son únicamente depuradores, como por ejemplo pdb de Python.

Aunque lo más común es encontrarlos dentro de los entornos de desarrollo, existen programas que son únicamente depuradores, como por ejemplo pdb de Python.

Solución

1. Incorrecto
2. Opción correcta

3.2.- Manejo de errores.


Dibujo de un signo de interrogación sobre una hoja de papel.

Cuando nuestro programa falla, se genera un error en forma de excepción. Las excepciones son errores detectados por el intérprete durante la ejecución del programa. Por ejemplo, podemos intentar dividir por cero o acceder a un archivo que no existe, entonces el programa genera una excepción avisando al usuario de que se ha producido un error. Nuestra labor como programadores es escribir el código de manera que se contemplen esas posibles excepciones, y se actúe en consecuencia. Si nuestro programa no contempla el manejo de excepciones, cuando ocurra alguna lo que pasará es que se generará el error y el programa dejará de ejecutarse o no tendrá el funcionamiento esperado.

En Python se utiliza una construcción **try-except** para capturar y tratar las excepciones. Dentro del bloque **try** se sitúa el código que creemos que podría producir una excepción, y dentro del bloque **except**, escribimos el código que se ejecutará si se produce dicha excepción.

Veamos un pequeño programa que lanzaría una excepción al intentar hacer una división entre 0:

```
def dividir(a, b):
    return a / b
dividir(3,0)
```

Si lo ejecutamos en el intérprete obtenemos la siguiente salida de error:

```
ZeroDivisionError: integer division or modulo by zero
```

Lo que nos dice el mensaje es que es una excepción, **ZeroDivisionError**, y la descripción del error: "**integer division or modulo by zero**" (módulo o división entera entre cero).

Si escribimos el mismo programa, pero teniendo en cuenta el manejo de excepciones, el código quedaría como sigue:

```
try:
    def dividir(a, b):
        return a / b
    dividir(1,0)
except:
    print ("Ha ocurrido un error")
```

Para saber más

Si quieres aprender más sobre excepciones puedes consultar el siguiente tutorial de Python:

Información sobre excepciones en Python.

4.- Creación de módulos en Odoo.

El desarrollo de módulos en un sistema ERP requiere mucha práctica y experiencia. En esta apartado intentaremos plasmar en este apartado los pasos más precisos para conseguirlo, sentar una base sobre la que trabajar y practicar.

Crear una aplicación en Odoo es un proceso ordenado que debe seguir varios pasos:

1. Análisis tecnológico inicial.
2. Desarrollar una serie de modelos de datos
3. Mapear los modelos de datos en Odoo.
4. Crear vistas y menús para visualizar los modelos de datos.

Pasos que se detallan en apartados a continuación.

Además, la abstracción conceptual que implica explicar conceptos como módulo, vistas, menús… de un sistema ERP, hace recomendable que todas las explicaciones se apoyen en ejemplos para facilitar su entendimiento.

Como apoyo a las explicaciones del presente apartado, todo concepto se ejemplificará con el desarrollo de un módulo para la **gestión de una Biblioteca.**

4.1.- PASO 1: análisis tecnológico inicial.

En este primer paso, se debe hacer un análisis tecnológico del módulo a llevar a cabo. Como se ha comentado, el módulo con el que se ejemplifican los conceptos de este apartado se basa en un módulo para la gestión de una Biblioteca.

Para articular este análisis, hemos de responder a **2 preguntas:**

- **¿Qué queremos que haga nuestra aplicación Odoo?**
- **¿Cómo estructuramos la información?**

Para nuestro ejemplo concreto, la respuesta a la **primera pregunta** sería fácil y directa: la idea es que el sistema ERP pueda contar con una aplicación en Odoo que permita dar de alta Libros, poder categorizarlos y gestionar los autores de los mismos.

Respecto a la **segunda pregunta**, hemos de profundizar en qué elementos son imprescendibles a disponer y qué información necesitamos de cada uno de ellos.

Para nuestra aplicación de Biblioteca, partimos de la premisa que un libro podrá tener una categoría y un autor principal y, a su vez, el autor podrá tener uno o varios libros asignados.

De esta forma, de cada **libro** se necesita recoger los siguientes datos

- nombre
- descripción
- fecha de publicación
- isbn
- imagen
- categoría a la que pertenece
- Autor

Las **categorías** deberán tener:

- Nombre
- código

Y de los **autores**:

- Nombre
- País
- Ciudad de nacimiento
- Colección de libros

Debes conocer

Es positivo también en este punto, si vamos también “pensando en la base de datos”, es decir, identificar qué tipo de dato almacenará cada uno de los campos.

Por ejemplo, para libro su tabla sería:

Campo	Tipo	Obs
nombre	Char	
descripcion	Text	
fecha_publicacion	Date	
isbn	Char	longitud 9 dígitos
imagen	Binary	
category_id	Many2one	Relación con Categoría
autor_id	Many2one	Relación con autor

Para categoría:

Campo	Tipo	Obs
nombre	Char	
codigo	Char	

Y para autor:

Campo	Tipo	Obs
nombre	Char	
pais	Text	
ciudad	Date	
libro_ids	One2many	Relación con modelo Libro

Sobre los tipos, cabe comentar:

- **One2many:** recordemos que se trata de un tipo de campo especial para crear relaciones entre modelos, en este caso un autor puede tener 1 o varios libros asignados.
- **Binary:** utilizado para almacenar datos binarios, ficheros, en este caso lo usamos para guardar las imágenes de los libros.

4.2.- PASO 2: desarrollar una serie de modelos de datos.

Para crear los modelos de datos se utiliza el lenguaje de programación Python. Modelos de datos que posteriormente se pueden integrar en Odoo. Modelos de datos que deben ser parte de un módulo.
Este proceso de creación de modelos de datos se divide en **3 pasos:**

1. Crear un módulo vacío.
2. Edición inicial fichero del módulo.
3. Creación de modelos Python.

Pasos que se detallan en subapartados a continuación.

Autoevaluación

Pregunta

El lenguaje de programación Python es el utilizado por sistemas ERP como Odoo.

Respuestas

Verdadero.

Falso.

Retroalimentación

Correcto. Requieren programas tanto par la generación del archivo como para el procesamiento posterior.

Incorrecto. Requieren programas tanto par la generación del archivo como para el procesamiento posterior.

Solución

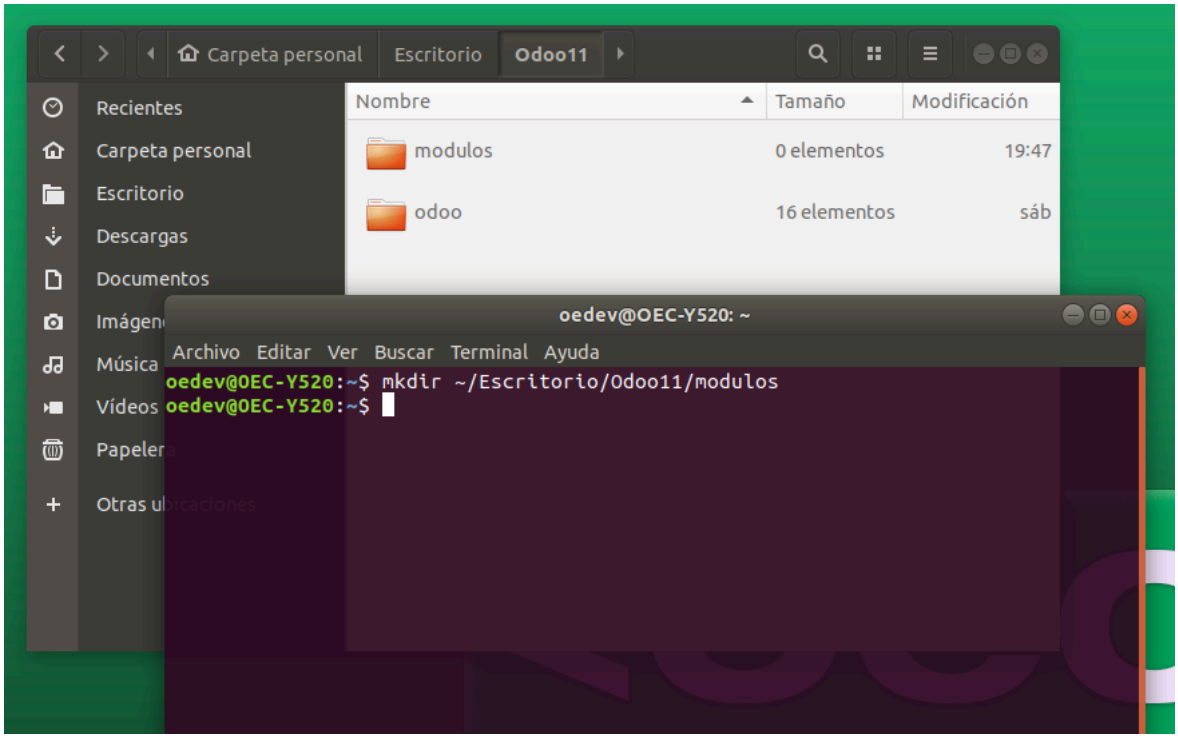
1. Opción correcta
2. Incorrecto

4.2.1.- Crear un módulo vacío.

Odoo nos permite crear un esqueleto para empezar el desarrollo de un módulo, para tal fin se utiliza el parámetro **scaffold**. Este esqueleto se compone de una serie de archivos base, archivos que tendremos que complementar para crear nuestro módulo.

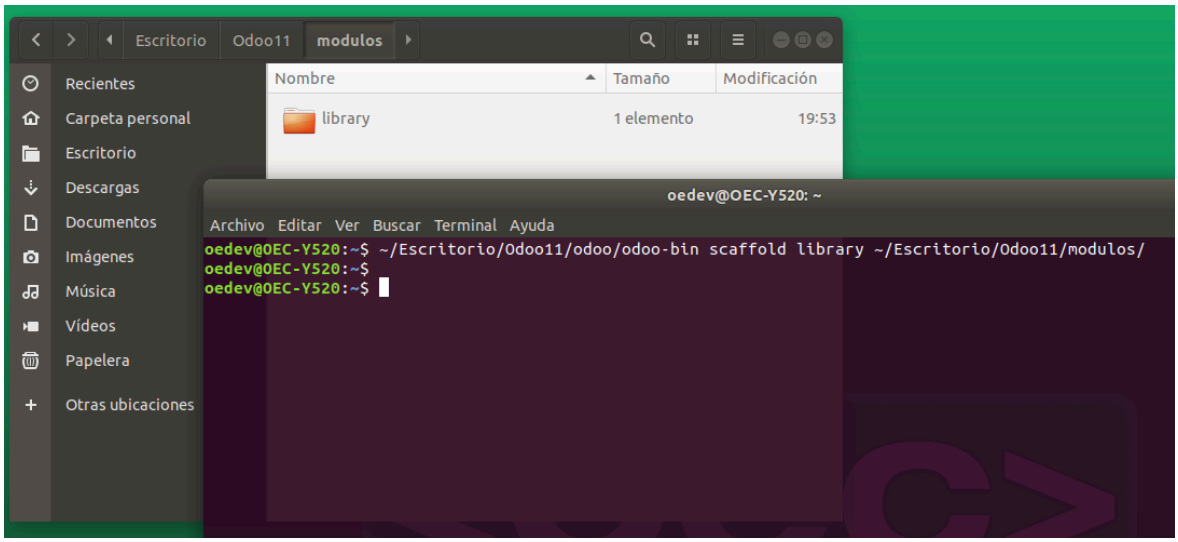
Lo ideal que creemos un directorio en el lugar donde estemos trabajando para incluirlo.

mkdir ~/Escritorio/Odoo11/modulos



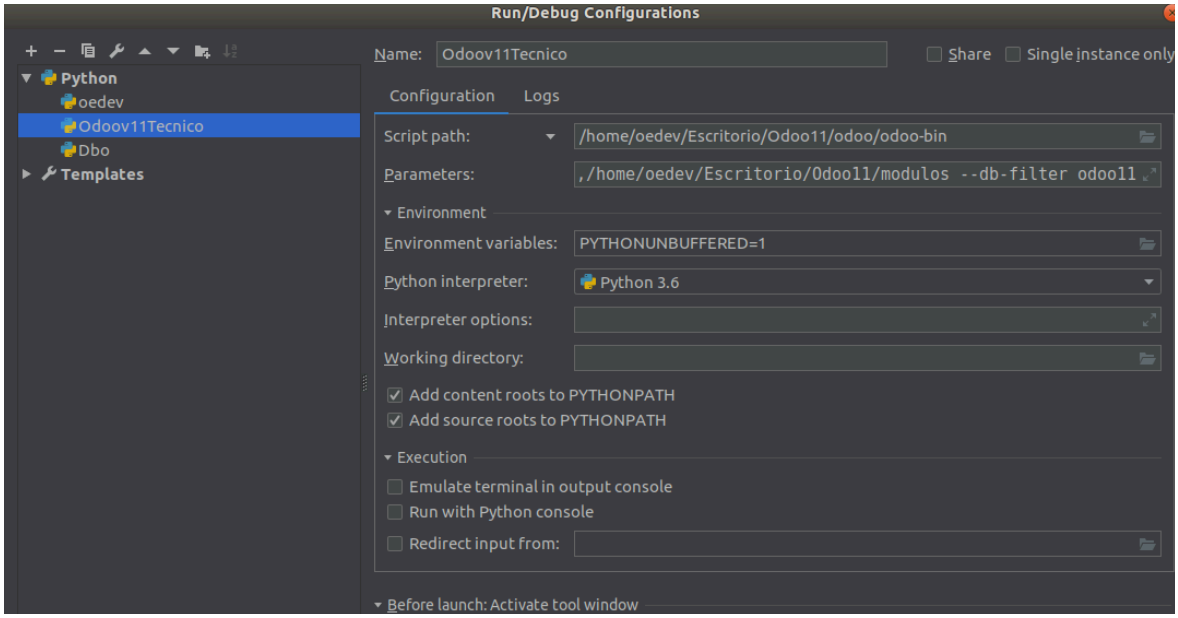
Y en el mismo, ejecutar el comando scaffold:

~/Escritorio/Odoo11/odoo/odoo-bin scaffold library ~/Escritorio/Odoo11/modulos/



Este módulo vacío lo hemos de editar. Módulo que como podemos comprobar,
está desarrollado en lenguaje de programación Python. Por tanto, hemos de editarlo en dicho lenguaje de programación.

Para facilitar esta edición, vamos a vincular el IDE (en nuestro caso, Pycharm) sobre este módulo vacío que acabamos de crear. De esta forma, podremos aprovechar hora sobre este módulo, hemos de vincular el IDE. En el caso de las capturas de ejemplo, el directorio a abrir por PyCharm es **~/Escritorio/Odoo11/odoo.**



En parameters se incluye:
path=/home/oedev/Escritorio/Odoo11/odoo/addons,/home/oedev/Escritorio/Odoo11/modulos --db-filter odoo11

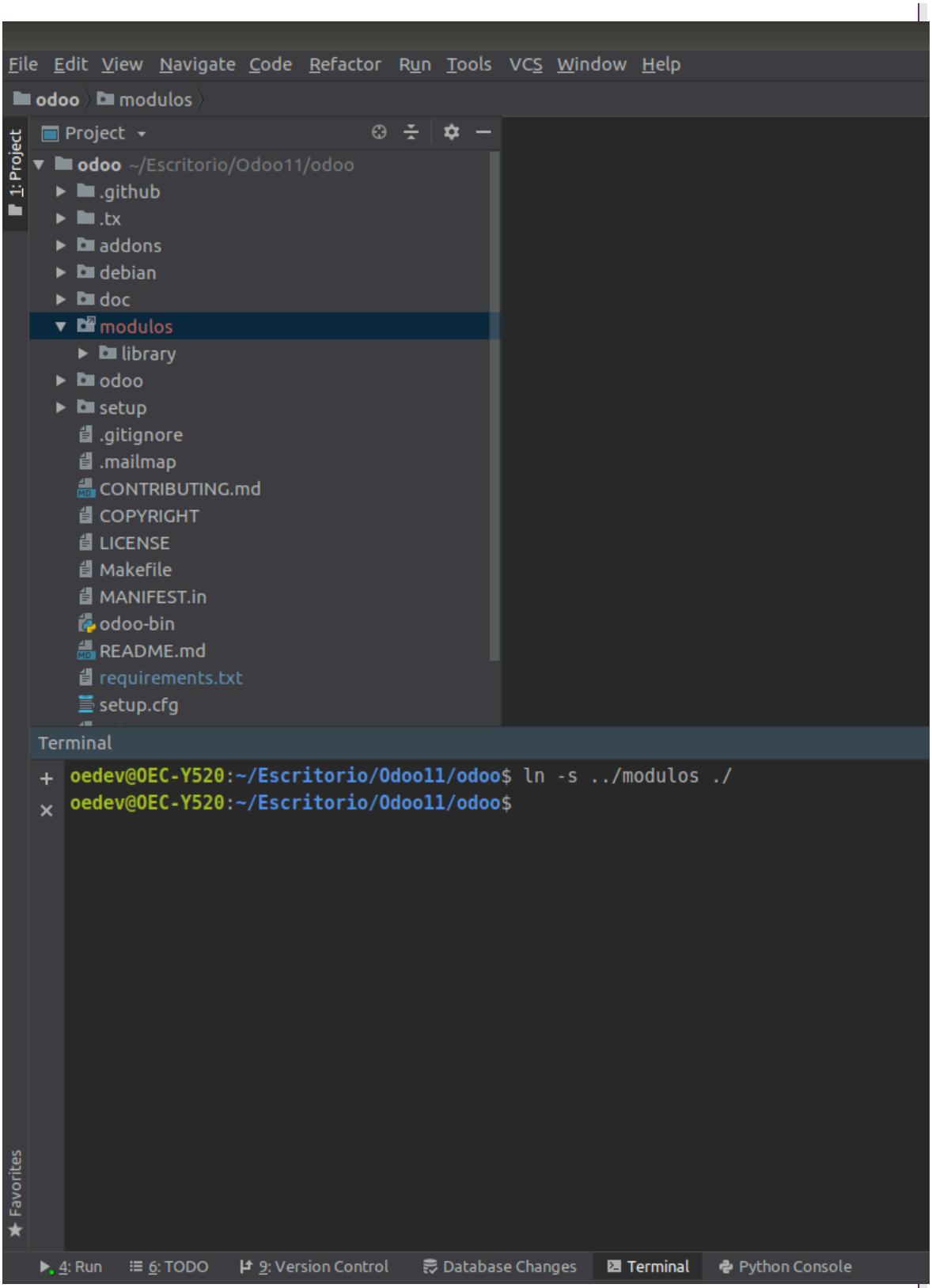
Donde "**oedev**" debe ser el nombre de usuario que tengamos en nuestro equipo.

.

Debes conocer

Desde el terminal de Pycharm, se puede crear crear un enlace simbólico / acceso directo al directorio módulos dentro de PyCharm. De esa manera podemos desarrollar nuestra App cómodamente desde el editor

(ln -s ../modulos ./)



Autoevaluación

Pregunta

El parámetro scaffold es el primer paso para crear un módulo en Python

Respuestas

Verdadero.

Falso.

Retroalimentación

Correcto. Con el atributo model definimos si se trata de una acción o de una vista.

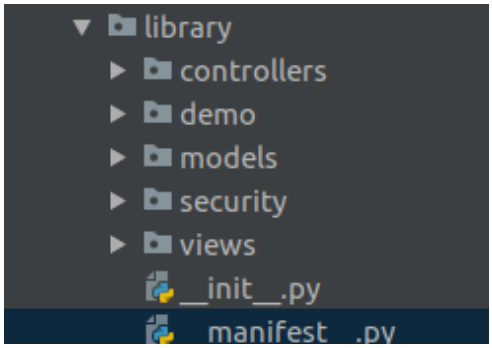
Incorrecto. Con el atributo model definimos si se trata de una acción o de una vista.

Solución

1. Opción correcta
2. Incorrecto

4.2.2.- Edición inicial ficheros del módulo.

Ya hemos creado nuestro primer módulo (eso si, vacío) gracias al parámetro scaffold. Ahora es importante reconocer los elementos de los que se compone el módulo que acabamos de crear, concretamente son los siguientes:



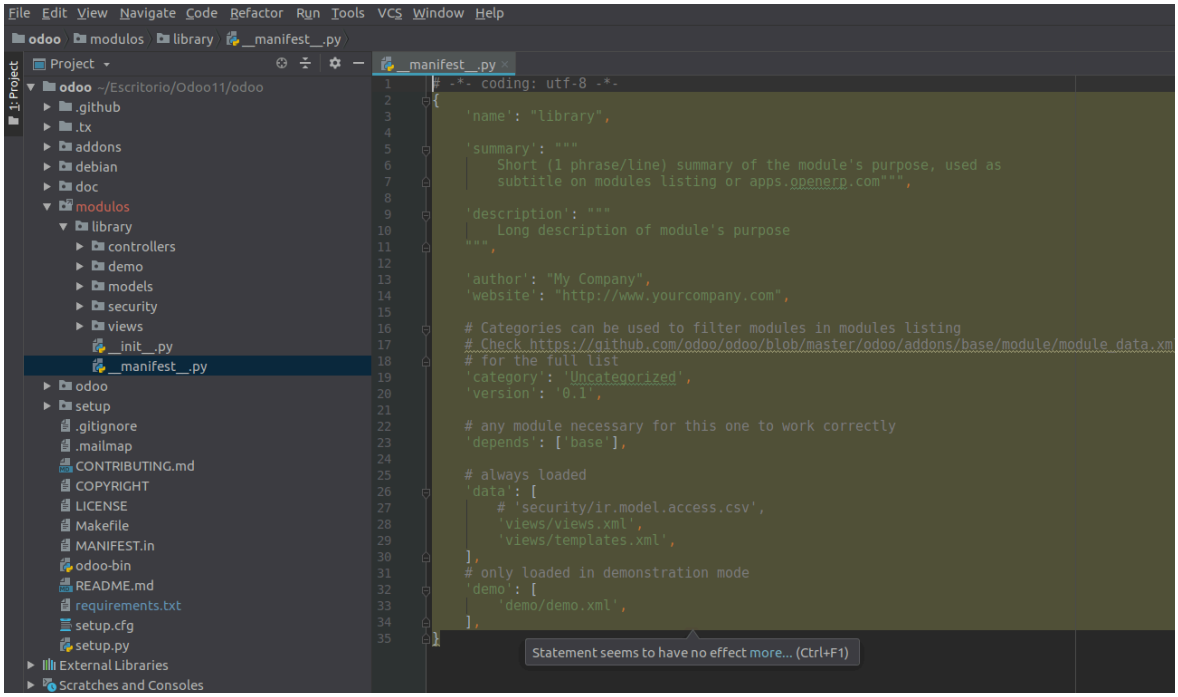
Estos elementos son:

- controllers: controladores para las peticiones web. No será necesario utilizarlo para el desarrollo de los contenidos de esta asignatura.
- demo: se utiliza cuando queremos usar datos de demostración. En nuestro caso, como podremos utilizar una base de datos en modo demo (es decir, con datos precargados) no será necesario utilizarlo.
- models: contiene todos los modelos Python
- security: ficheros de seguridad y acceso a los modelos. Tampoco será necesario utilizar para el desarrollo de los contenidos.
- views: aqui se incluyen las diferentes vistas, en formato XML, de los modelos a utilizar.
- __init__.py: fichero que le indica a python los modelos python / directorios a importar o indexar
- __manifest__.py: archivo de configuración principal.

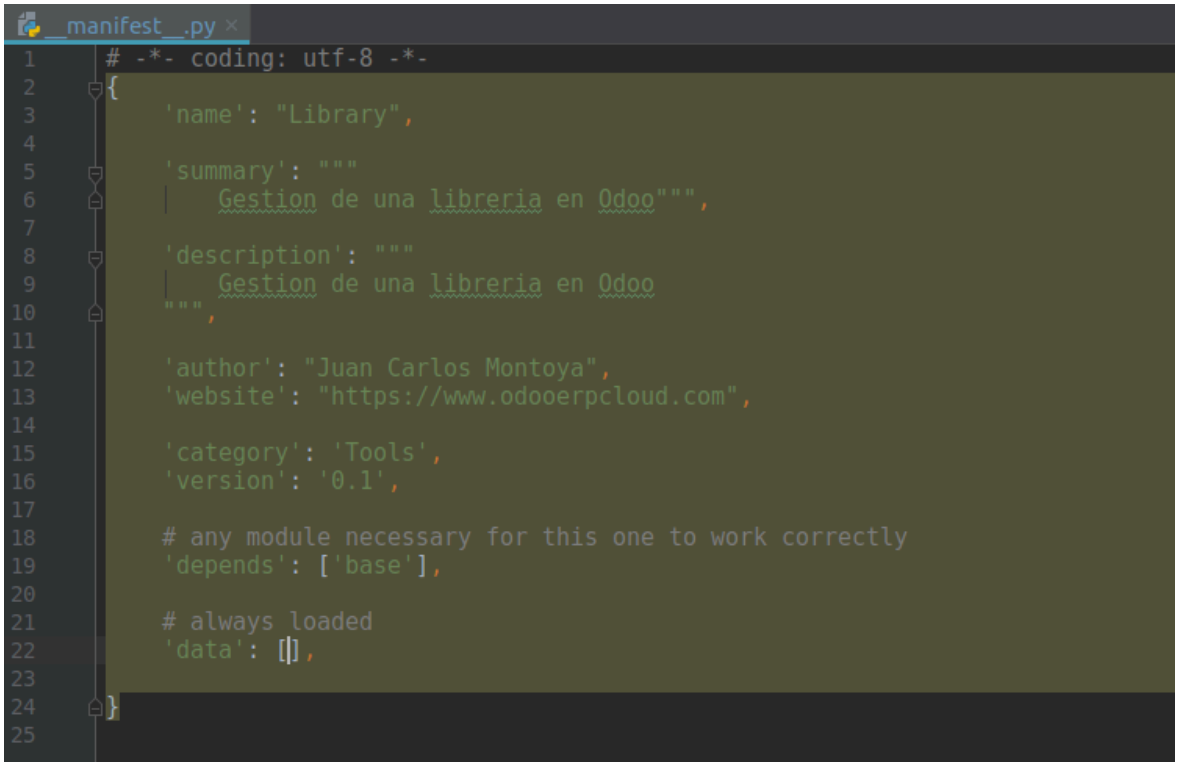
El fichero de un módulo Odoo es el __manifest__.py. El mismo contiene toda la metainformación tal como:

- Nombre módulo
- Autor
- Dependencias
- Datos / vistas a cargar
- Etc

Un archivo __manifest__.py podría entenderse como la “hoja de ruta” interna de un módulo. Archivo que define el módulo y llama al resto de archivos que forman parte del módulo. Al abrir el fichero con el editor lo encontramos así:



Tendríamos que editarlo básicamente para alinearlo con nuestro objetivo de desarrollo (gestión de biblioteca/librería):

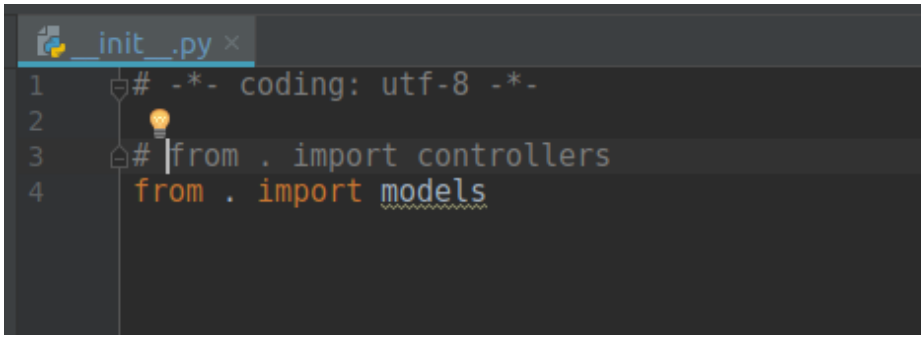


Los parámetros en “author” y “website” deben ser individuales para cada usuario.

Por otro lado, y como se ha comentado brevemente, para el desarrollo de los contenidos de este tema no es necesario utilizar el fichero Controllers.

Es por ello que vamos a modificar el fichero __init__ principal para que no utilice/importe el directorio controllers.

Para tal fin, nos basta con agregar un # delante para que python ignore esta línea.



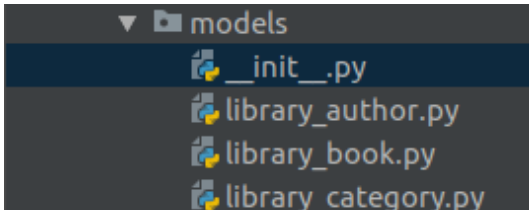
Para saber más

El carácter # es el carácter que se utiliza para agregar comentarios en Python. En el siguiente enlace puedes conocer más sobre los comentarios en Python:

Link tutorial comentarios Python

4.2.3.- Creación de modelos Python.

Llegados a este punto, podemos entrar en materia y crear los modelos de datos en Python. Crearemos 3, uno por cada modelo que necesitamos, es decir: library_author.py, library_category.py y library_book.py



La creación en el IDE es muy intuitiva y se asemeja a la creación de cualquier archivo en prácticamente cualquier programa.

Cuando los tengamos creados, hemos de importarlos en el fichero __init__.py que estan dentro de la carpeta models. Para tal fin:

```python
# -*- coding: utf-8 -*-

from . import library_author
from . import library_book
from . import library_category
```

De esta forma los modelos ya estarían creados y enlazados. Es el momento de definir los mismos. Para tal fin, apoyados en la sintaxis de Python, vamos a dar forma a los diferentes campos que tiene que incluir cada modelo, siempre siguiendo la estructura definida en el paso anterior de análisis tecnológico inicial.

De esta forma, el modelo de datos para Libro sería:

```python
# -*- coding: utf-8 -*-

from odoo import models, fields


class LibraryBook(models.Model):
    _name = "library.book"

    name = fields.Char(string="Name")
    description = fields.Text(string="Description")
    fecha_publicacion = fields.Date("Fecha Publicacion")
    isbn = fields.Char(string="ISBN", size=9)
    imagen = fields.Binary("Image")
    category_id = fields.Many2one("library.category", string="Categoria")
    autor_id = fields.Many2one("library.author", string="Autor")
```

El de Categoría:

library_category.py

```python
# -*- coding: utf-8 -*-

from odoo import models, fields


class LibraryCategory(models.Model):
    _name = "library.category"

    name = fields.Char(string="Name")
    codigo = fields.Char(string="Codigo")

```

Y el de Autor:

library_author.py

```python
# -*- coding: utf-8 -*-

from odoo import models, fields


class LibraryAuthor(models.Model):
    _name = "library.author"

    name = fields.Char(string="Nombre")
    pais = fields.Many2one("res.country", string="Pais")
    ciudad = fields.Char(string="Ciudad")
    libro_ids = fields.One2many(
        "library.book", "autor_id", string="Libros"
    )

```

4.3.- PASO 3: mapear los modelos de datos en Odoo.

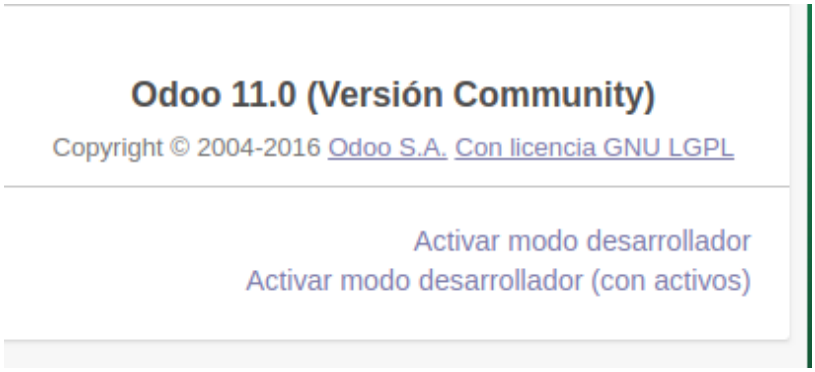
Llegados a este punto, ya tenemos el módulo con los modelos creados en Python, es momento de instalarlo en el sistema ERP, de mapearlo en Odoo.

El mapeado de los modelos de datos consiste en un proceso consistente en carga de información en el sistema ERP, con el fin que automáticamente el ERP transforme estos modelos de datos en tablas SQL.

Tablas SQL que, al tratarse del sistema Odoo que hemos venido desarrollando en las prácticas, utilizarán la tecnología PostgreSQL como sistema gestor de base de datos.

Gracias a Odoo, este proceso sin duda es bastante sencillo y automático. Odoo se apoya en un módulo llamado Library, el cual hemos de instalar. Para tal fin:

1. Reiniciamos el servicio de odoo
2. En "Ajustes", activamos el modo desarrollador

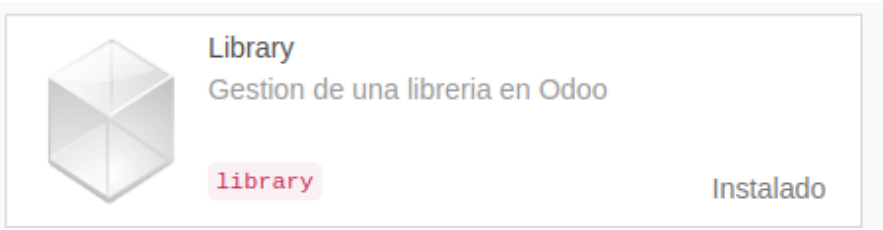


3. Desde el menú "Aplicaciones", seleccionamos la opción de actualizar lista de aplicaciones:



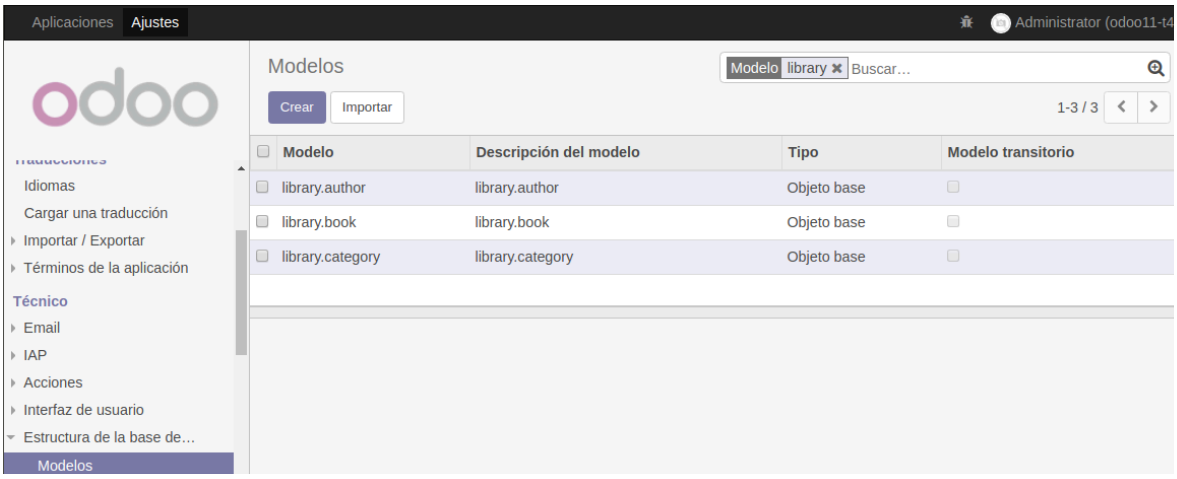
4. Utilizamos el buscador para localizar nuestro módulo library y lo instalamos:

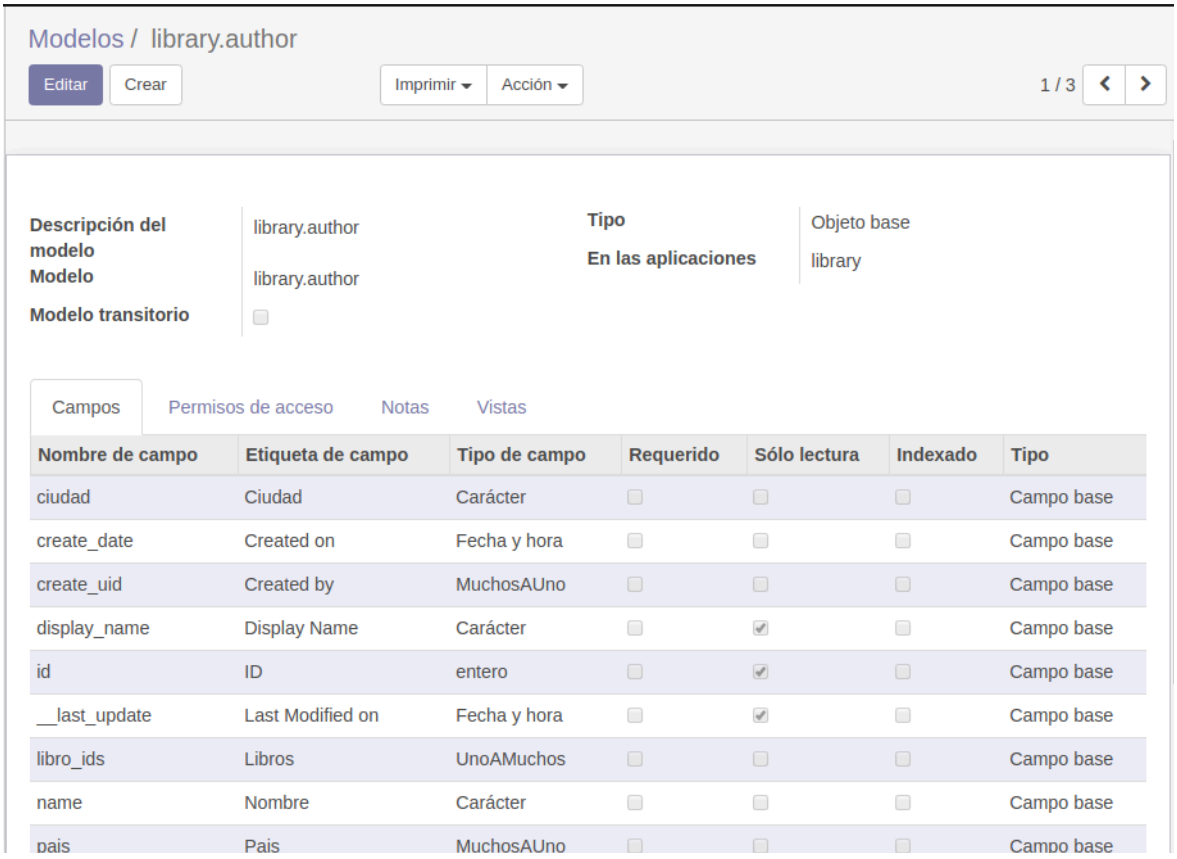




Una vez instalado, podremos comprobar con se han mapeado los modelos de datos Python en Postgresql, es decir, se han creado tablas SQL en Postgresql con los mismos.

Accediendo a Ajustes/Técnico/Estructura de la base de datos/Modelos y buscamos buscamos por la palabra library, vemos que ya existen nuestros modelos

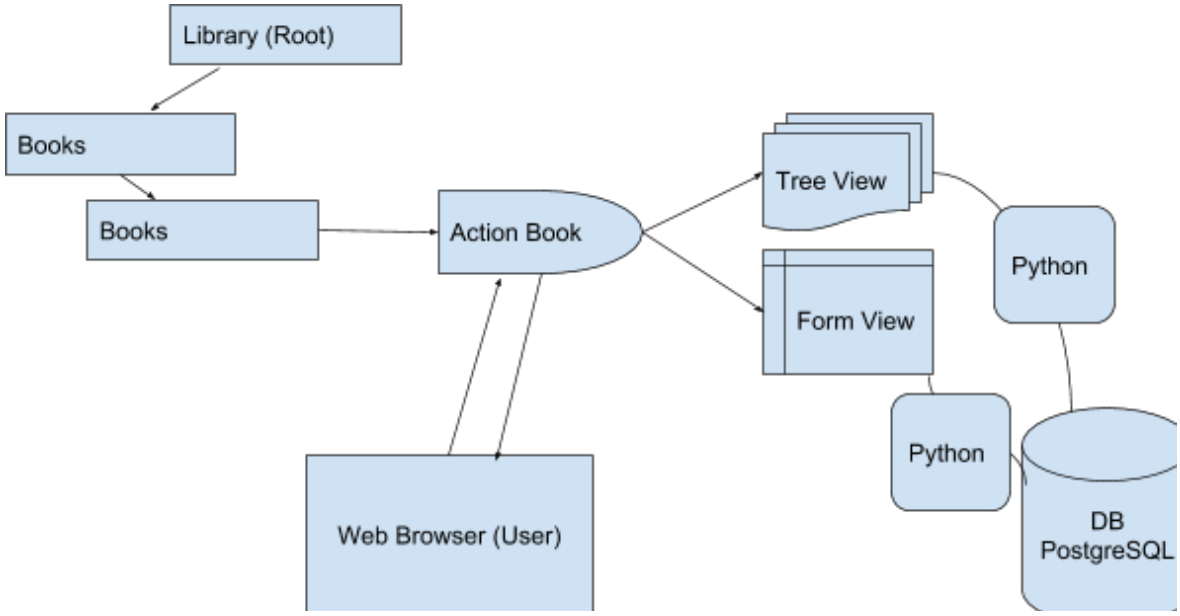




4.4.- PASO 4: crear vistas y menús para visualizar los modelos de datos.

Aunque ya hayamos mapeado los modelos en Odoo, creados previamente con Python, esto no quiere decir que podamos operar con ellos.

Para empezar a dar de alta libros, categorías y autores nos hace falta “la parte visual” de Odoo, y para eso tenemos que crear los menús y las vistas para acceder a ellos.
Como se ha comentado, Odoo sigue el patrón MVC. De esta forma, en Odoo se separa en modelos de datos, vistas y controladores (Python) y por otro lado la capa de datos en Postgresql



La creación de vistas y menús se divide **en 4 pasos:**

1. Creación archivos XML
2. Completar ficheros XML
3. Crear una acción
4. Crear menús de acceso.

Pasos que detallaremos a continuación.

Debes conocer

La creación de vistas y menús en Odoo es un proceso del cual se pueden encontrar mucha documentación en Internet. Además es un proceso muy completo que, aunque se controle y sepa llevar a cabo, siempre se puede aprender algo nuevo del mismo.

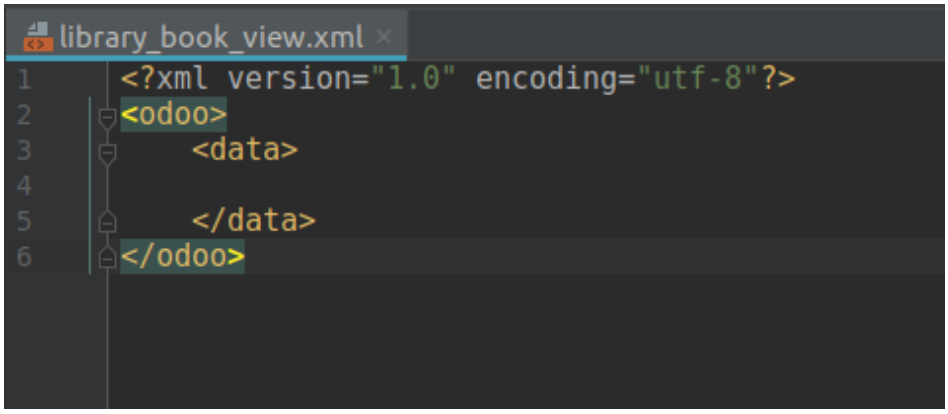
Destaca especialmente el gran número de vídeo-tutoriales que existen en Youtube. Se recomienda poner "Vistas y Menús Odoo" en el buscador de

Youtube y analizar la información obtenida.

4.4.1.- Creación archivos XML.

Para el desarrollo de aplicaciones en Odoo, las vistas se crean en un fichero XML, por lo que hemos de crear un XML para cada modelo de datos manteniendo el estándar de Odoo en el nombramiento de ficheros, es decir, nombre_modelo_view.xml
Estos ficheros XML pueden crearse directamentes desde el IDE, concretamente en la carpeta views del módulo. Esta carpeta podrá tener uno o varios ficheros de ejemplo, los mismos se podrán eliminar o reutilizar.

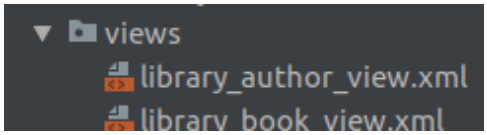
Estos ficheros XML deben tener obligatoriamente la siguiente estructura:



Donde:

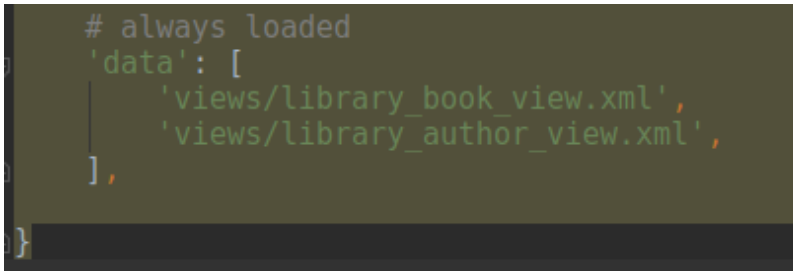
- La primera línea declara que es un fichero XML
- La etiqueta odoo y data son elemento raíz obligatorio en la definición de vistas en Odoo.

Siguiendo el trabajo en el desarrollo de nuestro filmoteca, vamos a crear 2 vistas una para libros y otra para autores, es decir 2 ficheros XML:



Además, estas vistas deben ser agregadas en el diccionario del fichero __manifest__.py

en “data”_



Anexo.- Licencias de recursos.

Ningún recurso de fuentes externas que requiera citar explícitamente sus datos de licencia ha sido usado en esta unidad, por lo que este anexo queda vacío. Todos los recursos utilizados, de fuentes internas, se acogen al Aviso Legal de la plataforma.